

claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf_b531eaeef-39c\agent-ac34cfbf245fe7b81.jsonl`
- SHA-256: `313bc914e96d118a980097b93d73103bdd782cf25e1ce886a97533a587d4912e`
- Source modified: `2026-06-19T16:58:03+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_b531eaeef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`
```

Transcript

1. user (2026-06-19T16:57:07.029Z)

You are verifying claims for a design doc; the doc must only assert what the code supports. Verify: the raw detector candidate cache `det_raw.jsonl` PERSISTS after a successful run (is NOT deleted), so the full multi-candidate set is available offline for replay with no GPU. Check `backend/pipeline/detectors/wasb_runner.py` (what cleanup deletes on success) and `wasb_infer.py` (cache location/keying/format). Document the on-disk JSONL record format. Cite line numbers. VERIFIED/REFUTED/PARTIAL.

Read the actual files (do not guess). Return the structured verdict.

2. assistant (2026-06-19T16:57:10.039Z)

I'll verify the claim about the raw detector candidate cache persistence. Let me read the two relevant files.

3. user (2026-06-19T16:57:10.983Z)

```
1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4          1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5          2. copies wasb_infer.py into the WASB repo's src/ dir,
6          3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7          4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21     TrackNetRunner.
22     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
24     wsl_tilde_quote
25
26     logger = logging.getLogger(__name__)
27
28     # Windows path to the inference wrapper we stage into WSL.
```

```

27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45         # extraction that dominates a long clip. Output is bit-identical to the PNG path
46         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47         stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                  f"python -c \"import torch; print('torch', torch.__version__, "
76                  f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"

```

```

86         return True, out
87
88     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89         src_mnt = to_wsl_mnt_path(video_win_path)
90         reason = self.wsl_source_unreadable_reason(src_mnt)
91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104 res.stdout).strip())
105             return None
106         return dest
107
108     def _stage_infer_script(self) -> bool:
109         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110         src_mnt = to_wsl_mnt_path(_INFER_WIN)
111         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
112 OK"
113         res = self._wsl_bash(cmd)
114         return res.returncode == 0 and "OK" in (res.stdout or "")
115
116     def run_predict(self, video_win_path: str, output_win_dir: str,
117 video_id: Optional[str] = None) -> Optional[str]:
118         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122         can never reuse each other's staged frames, resumable cache, or CSV.
123
124         """
125         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
126         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
127         # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
128         # the CSV landed inside WSL while Windows polled the repo-relative path.
129         output_win_dir = os.path.abspath(output_win_dir)
130         os.makedirs(output_win_dir, exist_ok=True)
131         # Normalize for cross-platform basename (tests use Windows paths on Linux).
132         video_win_path = str(video_win_path).replace("\\", "/")
133         base = os.path.basename(video_win_path)
134         stem, ext = os.path.splitext(base)
135         # Unique, content-derived key: same filename + different bytes -> different key.
136         key = f"{stem}__{video_id[:12]}" if video_id else stem
137         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
138         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
139
140         if not self._stage_infer_script():
141             logger.error("Could not stage wasb_infer.py into WSL.")
142             return None
143         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
144         if staged_video is None:
145             return None
146         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
147
148         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
149         # WSL stage filesystem is already low so the user can clean up before it fills.
150         # (Streaming never materializes the PNGs, so the guard only matters off the fast
151         path.)

```

```

147         if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
149             if free is not None and free < self.cfg.min_free_gb:
150                 logger.warning(
151                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
152                         "may fill the disk ? consider running tools/cleanup_caches.py.",
153                         free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157         if self.cfg.stream_video:
158             frame_args = "--stream-video "
159         else:
160             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161         cmd = (
162             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163             f"python wasb_infer.py "
164             f"--video {wsl_tilde_quote(staged_video)} "
165             f"{frame_args}"
166             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167             f"--sport {shlex.quote(self.cfg.sport)} "
168             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}}")
169
170         logger.info("Running WASB inference on %s ...", base)
171         try:
172             res = self._wsl_bash(cmd)
173         except subprocess.TimeoutExpired:
174             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175             return None
176         if res.returncode != 0:
177             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178             return None
179         if not os.path.exists(expected_csv_win):
180             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181             return None
182         logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184         # Success ? the trajectory CSV is the durable output; the staged video copy and
185         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187         # so the frame cache + detector cache remain for a resumable re-run.
188         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189         if not self.cfg.keep_frames:
190             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191             logger.info("Cache hygiene: removing staged video%s for %s "
192                         "(set indexing.wasb.keep_frames=true to retain).",
193                         "" if self.cfg.stream_video else " + frame cache", key)
194             self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

```

1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
37         ../pretrained_weights/wasb_badminton_best.pth.tar \
38         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39         every 1000] [--fresh]
40
41     """
42     import os
43     import os.path as osp
44     import csv
45     import glob
46     import json
47     import argparse
48     import logging
49     from collections import defaultdict
50     from contextlib import contextmanager
51
52     logger = logging.getLogger(__name__)
53
54     CACHE_VERSION = 1
55     _DET_FILE = "det_raw.jsonl"
56     _MANIFEST_FILE = "manifest.json"
57
58     def _build_cfg(weights: str, sport: str):
59         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
60         from hydra import compose, initialize
61         with initialize(config_path="configs", version_base=None):
62             cfg = compose(config_name="eval", overrides=[
63                 f"dataset={sport}",
64                 "model=wasb",
65                 f"detector.model_path={weights}",

```

```

64         "runner.device=cuda",
65         "runner.gpus=[0]",          # this box has a single GPU; default config assumes
more
66     ])
67     return cfg
68
69
70     def _frame_id(path: str) -> int:
71         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
72         stem = osp.splitext(osp.basename(path))[0]
73         digits = "".join(ch for ch in stem if ch.isdigit())
74         return int(digits) if digits else -1
75
76
77     # ----- #
78     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79     # ----- #
80     def _cache_paths(cache_dir: str):
81         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84     def default_cache_dir(out_csv: str) -> str:
85         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86         d = osp.dirname(osp.abspath(out_csv))
87         stem = osp.splitext(osp.basename(out_csv))[0]
88         return osp.join(d, stem + "_wasbcache")
89
90
91     def _atomic_write_text(path: str, text: str) -> None:
92         """Write then rename so a crash mid-write can't leave a half-written file."""
93         tmp = path + ".tmp"
94         with open(tmp, "w") as f:
95             f.write(text)
96             f.flush()
97             os.fsync(f.fileno())
98         os.replace(tmp, path)
99
100
101     def _det_plain(det) -> list:
102         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
103         xy = det["xy"]
104         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
105
106
107     def _dets_to_tracker(plain_list):
108         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
109         the tracker does vector math / np.linalg.norm on it)."""
110         import numpy as np
111         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
112                 for p in plain_list]
113
114
115     def write_manifest(cache_dir: str, manifest: dict) -> None:
116         _, mpath = _cache_paths(cache_dir)
117         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120     def read_manifest(cache_dir: str):
121         _, mpath = _cache_paths(cache_dir)
122         if not osp.exists(mpath):
123             return None
124         try:

```

```

125         with open(mpath) as f:
126             return json.load(f)
127     except (json.JSONDecodeError, OSError):
128         return None
129
130
131     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
132                             frames_in: int, frames_total: int) -> bool:
133         """A cache is reusable only if the run that produced it matches this run's
134         inputs."""
135         if not manifest or manifest.get("version") != CACHE_VERSION:
136             return False
137         return (
138             manifest.get("frames_dir") == osp.abspath(frames_dir)
139             and manifest.get("weights") == weights
140             and manifest.get("sport") == sport
141             and manifest.get("frames_in") == frames_in
142             and manifest.get("frames_total") == frames_total
143         )
144
145     def reset_cache(cache_dir: str) -> None:
146         """Drop any existing cache so the next run starts clean."""
147         det_jsonl, mpath = _cache_paths(cache_dir)
148         for p in (det_jsonl, mpath):
149             if osp.exists(p):
150                 os.remove(p)
151
152
153     def load_and_clean_cache(cache_dir: str):
154         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
156         prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159         [x, y, score, scale] candidates (accumulated across every window the frame is in,
160         in window order ? identical to a non-resumed run).
161         """
162         det_jsonl, _ = _cache_paths(cache_dir)
163         det_by_fid = defaultdict(list)
164         valid: list = []
165         if osp.exists(det_jsonl):
166             with open(det_jsonl, "r") as f:
167                 for line in f:
168                     s = line.strip()
169                     if not s:
170                         continue
171                     try:
172                         obj = json.loads(s)
173                     except json.JSONDecodeError:
174                         break # truncated trailing line (crash mid-
flush)
175                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
176                         break
177                     valid.append(s)
178                     for fid, plain in obj["f"]:
179                         det_by_fid[int(fid)].extend([list(p) for p in plain])
180                 # Guarantee a clean append boundary by rewriting only the good prefix.
181                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
182         return len(valid), det_by_fid
183
184
185     def _append_windows(fh, objs) -> None:
186         """Append window records as JSONL and durably flush (bounded loss on crash)."""
187         for o in objs:

```

```

188         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
189     fh.flush()
190     os.fsync(fh.fileno())
191
192
193 def _atomic_write_trajectory(out_csv: str, rows) -> None:
194     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
195     tmp = out_csv + ".tmp"
196     with open(tmp, "w", newline="") as f:
197         w = csv.writer(f)
198         w.writerow(["Frame", "Visibility", "X", "Y"])
199         w.writerows(rows)
200         f.flush()
201         os.fsync(f.fileno())
202     os.replace(tmp, out_csv)
203
204
205 # ----- #
206 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207 # ----- #
208 _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211 def synthetic_frame_path(index: int) -> str:
212     """Stable, index-encoding frame name used by the streaming path.
213
214     It mirrors the on-disk names ``extract_frames`` writes (``{:08d}.png``) so the
215     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
216     frames came from disk or from the in-memory stream. The streaming image loader
217     inverts this name back to an index to fetch the decoded frame.
218     """
219     return _STREAM_FRAME_FMT.format(int(index))
220
221
222 def build_windows(frames: list, frames_in: int):
223     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
224     + checkpoint). Pure list math, identical for the disk and streaming paths.
225
226     Returns a list of windows, each a list of ``frames_in`` consecutive paths
227     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
228     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
229     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
230     """
231     return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
232
233
234 class SequentialFrameStore:
235     """In-memory sliding buffer over a forward-only frame reader, sized for the
236     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
237
238     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
239     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
240     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
241     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
242     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
243     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
244     earliest index any not-yet-finished window can still ask for) and decode each source
245     frame exactly once via the forward-only ``reader(index)``.
246     """
247
248     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
249         self._reader = reader
250         self._total = int(total)
251         self._window = max(1, int(window))
252         self._buf: dict = {}

```



```

253         # On a resumed run the detector's first needed frame is `start_index`; begin
254         # pulling there so the reader can skip (seek past) the already-cached prefix
255         # instead of decoding 0..start_index-1 just to throw them away.
256         self._next = int(start_index)          # next index not yet pulled from the reader
257         self._max_seen = int(start_index) - 1
258         self._floor = int(start_index)         # lowest index we still keep (never evict
below)
259
260     def get(self, index: int):
261         index = int(index)
262         if index < self._floor:
263             raise KeyError(
264                 f"frame {index} already evicted (below floor {self._floor}); the access
"
265                 f"pattern must stay within a window of the highest frame seen")
266         if index >= self._total:
267             raise KeyError(f"frame {index} out of range (total {self._total})")
268         # Pull forward from the reader until the requested index is buffered.
269         while self._next <= index:
270             self._buf[self._next] = self._reader(self._next)
271             self._next += 1
272         if index > self._max_seen:
273             self._max_seen = index
274         # Evict frames older than the earliest a still-running window could re-request:
275         # once we've seen index `m`, no future window starts before `m - (window-1)`.
276         new_floor = max(self._floor, self._max_seen - (self._window - 1))
277         for k in range(self._floor, new_floor):
278             self._buf.pop(k, None)
279         self._floor = new_floor
280         return self._buf[index]
281
282
283     def _index_from_synthetic_path(path: str) -> int:
284         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287         in lockstep with the on-disk naming scheme.
288         """
289         return _frame_id(path)
290
291
292     def _bgr_to_pil_rgb(frame_bgr):
293         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
294         PIL RGB image WASB's disk path produces.
295
296         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
297         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
298         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
299         frame indistinguishable from the on-disk one to everything downstream.
300         """
301         import cv2
302         from PIL import Image
303         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
304
305
306     def _make_streamed_read_image(store, real_read_image):
307         """Build the ``read_image`` replacement used during a streaming detector pass.
308
309         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
310         matching ``read_image``'s real return type); any path that doesn't parse to an index

```

```

falls
311         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
312         without the WSL-only ``dataloaders`` package.
313         """
314         def _read_image(path, *args, **kwargs):
315             idx = _index_from_synthetic_path(path)
316             if idx < 0:
317                 return real_read_image(path, *args, **kwargs)
318             return _bgr_to_pil_rgb(store.get(idx))
319         return _read_image
320
321
322     @contextmanager
323     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
int = 0):
324         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
325         frames during the detector pass, byte-for-byte as it would over real PNGs.
326
327         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
328         ``cv2.imread``. ``read_image`` (``utils.utils``) does
329         ``Image.open(path).convert('RGB')``,
330         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
331         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
332         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
333         ``osp.exists`` guard is sidestepped for synthetic stream paths.
334
335         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
336         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
337         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
338         reference).
339         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
340         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
341         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
342         original reader is restored on exit.
343         """
344         from dataloaders import dataset_loader as _dl
345         store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
346         real_read_image = _dl.read_image
347         # Rebind read_image in the consuming module for the duration of the detector pass;
348         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
349         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
350         try:
351             yield store
352         finally:
353             _dl.read_image = real_read_image # type: ignore[assignment]
354
355
356     # ----- #
357     # Inference
358     # ----- #
359     def run(frames, weights: str, out_csv: str, sport: str = "badminton",

```

```

360         limit: int = 0, batch_size: int = 8, num_workers: int = 4,
361         log_every_batches: int = 50, cache_dir: "str | None" = None,
362         flush_every: int = 1000, fresh: bool = False,
363         frame_loader=None, source_key: "str | None" = None) -> str:
364     """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
365
366     Two equivalent ways to supply pixels (output is bit-identical between them):
367
368     * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369       globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
370       ``cv2.imread``.
371     * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
372       frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
373       pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
374       reader, which returns a PIL RGB image) over the DataLoader pass so every other
375       line of
376       ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
377       identical
378       to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
379       ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
380       ``num_workers=0``
381       (the in-process shim + buffer can't cross worker processes).
382
383     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
384     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
385     after reboot still matches.
386     """
387     import time
388     import torch
389     from torch.utils.data import DataLoader
390     from detectors import build_detector
391     from trackers import build_tracker
392     from dataloaders import build_img_transforms, build_seq_transforms
393     from dataloaders.dataset_loader import ImageDataset
394     from utils import Center
395
396     streaming = frame_loader is not None
397
398     cfg = _build_cfg(weights, sport)
399     frames_in = int(cfg["model"]["frames_in"])
400     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
401     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
402
403     if streaming:
404         # `frames` is already the ordered list of (synthetic) frame paths.
405         if not isinstance(frames, (list, tuple)):
406             raise SystemExit("streaming mode requires `frames` to be a list of frame
407             paths")
408         frames = list(frames)
409         frames_dir = source_key or "stream"
410     else:
411         frames_dir = frames
412         frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
413         glob.glob(osp.join(frames_dir, "*.jpg"))
414         if limit:
415             frames = frames[:limit]
416         if len(frames) < frames_in:
417             where = frames_dir if not streaming else "video stream"
418             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
419
420     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
421     checkpoint).
422     samples = []
423     for window in build_windows(frames, frames_in):
424         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}

```

```

for p in window]
418     samples.append({"frames": window, "annos": annos})
419     windows_total = len(samples)
420
421     # ---- cache setup / resume decision ----- #
422     if cache_dir is None:
423         cache_dir = default_cache_dir(out_csv)
424         os.makedirs(cache_dir, exist_ok=True)
425         det_jsonl, _ = _cache_paths(cache_dir)
426         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427         manifest = None if fresh else read_manifest(cache_dir)
428         resume = manifest_compatible(
429             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430             frames_in=frames_in, frames_total=len(frames),
431         )
432         if resume:
433             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435         else:
436             if manifest is not None:
437                 logger.info("cache incompatible with this run -> starting fresh")
438                 reset_cache(cache_dir)
439                 windows_done, det_by_fid = 0, defaultdict(list)
440
441         base_manifest = {
442             "version": CACHE_VERSION,
443             "frames_dir": cache_key,
444             "weights": weights,
445             "sport": sport,
446             "frames_in": frames_in,
447             "frames_total": len(frames),
448             "windows_total": windows_total,
449             "windows_done": windows_done,
450         }
451         write_manifest(cache_dir, base_manifest)
452
453     # ---- detector pass over the un-cached windows ----- #
454     remaining = samples[windows_done:]
455     if remaining:
456         _, transform_test = build_img_transforms(cfg)
457         try:
458             _, seq_transform_test = build_seq_transforms(cfg)
459         except Exception:
460             seq_transform_test = None
461         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462             transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
463         # Streaming forces single-process loading: the in-memory frame store + the
464         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
465         loader_workers = 0 if streaming else num_workers
466         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
467         detector = build_detector(cfg)
468
469         from contextlib import ExitStack
470
471         t0 = time.time()
472         done = windows_done
473         win_idx = windows_done
474         log_every = max(1, log_every_batches)
475         flush_n = max(1, flush_every)
476         pending = []
477         logger.info(f"detecting on {len(remaining)} remaining windows ")

```

```

478         f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
479         f"source={'video-stream' if streaming else 'frames-dir'})..."
480     det_f = open(det_jsonl, "a")
481     try:
482         with ExitStack() as stack:
483             stack.enter_context(torch.no_grad())
484             # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
485             # in-memory decoded frame for that synthetic path; every other line of
486             # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
487             # The detector's first needed frame is `windows_done` (window i starts
at
488             # frame i), so prime the store there for a resume.
489             if streaming:
490                 stack.enter_context(
491                     _streaming_read_image_patch(frame_loader, len(frames),
492                                                 window=frames_in,
start_index=windows_done))
493             for bi, batch in enumerate(loader):
494                 imgs, _hms, trans = batch[0], batch[1], batch[2]
495                 img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
496                 batch_results, _ = detector.run_tensor(imgs, trans)
497                 nb = imgs.shape[0]
498                 for ib in range(nb):
499                     frames_payload = []
500                     for ie in sorted(batch_results[ib].keys()):
501                         p = img_paths[ie][ib]
502                         fid = _frame_id(p)
503                         plain = [_det_plain(d) for d in batch_results[ib][ie]]
504                         frames_payload.append([fid, plain])
505                         det_by_fid[fid].extend(plain)
506                     pending.append({"w": win_idx, "f": frames_payload})
507                     win_idx += 1
508                 done += nb
509                 if len(pending) >= flush_n or done >= windows_total:
510                     _append_windows(det_f, pending)
511                     pending = []
512                     base_manifest["windows_done"] = done
513                     write_manifest(cache_dir, base_manifest)
514                 if (bi + 1) % log_every == 0 or done >= windows_total:
515                     el = time.time() - t0
516                     new_done = done - windows_done
517                     rate = new_done / el if el > 0 else 0.0
518                     eta = (windows_total - done) / rate if rate > 0 else 0.0
519                     logger.info(f"{done}/{windows_total} "
520                                f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
521                                f"elapsed {el:.0f}s | ETA {eta:.0f}s")
522                 if pending:
523                     _append_windows(det_f, pending)
524                     base_manifest["windows_done"] = done
525                     write_manifest(cache_dir, base_manifest)
526             finally:
527                 det_f.close()
528                 logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529         else:
530             logger.info("detector cache already complete -> tracker-only re-run")
531
532     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533     tracker = build_tracker(cfg)
534     tracker.refresh()
535     rows = []

```

```

536     for fid in sorted(det_by_fid.keys()):
537         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539
540     _atomic_write_trajectory(out_csv, rows)
541     visible = sum(1 for _, v, *_ in rows if v)
542     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
543     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
544     return out_csv
545
546
547 def extract_frames(video_path: str, frames_dir: str) -> str:
548     """Decode a video to PNG frames named by source frame index (00000000.png ...).
549
550     Idempotent: if the dir already holds the expected number of frames (matching the
551     container's reported frame count) we skip re-decoding ? re-extraction after a
552     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
553     extracting with ffmpeg on the host and passing --frames_dir.
554     """
555     import cv2
556     os.makedirs(frames_dir, exist_ok=True)
557     cap = cv2.VideoCapture(video_path)
558     if not cap.isOpened():
559         raise SystemExit(f"cannot open video: {video_path}")
560     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
561     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
562     if expected > 0 and existing >= expected:
563         cap.release()
564         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
565         return frames_dir
566     i = 0
567     while True:
568         ok, frame = cap.read()
569         if not ok:
570             break
571         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
572         i += 1
573     cap.release()
574     logger.info(f"extracted {i} frames -> {frames_dir}")
575     return frames_dir
576
577
578 def _probe_frame_count(video_path: str) -> int:
579     """Container-reported frame count via cv2 (0 if unknown). Used to size the
580     stream."""
581     import cv2
582     cap = cv2.VideoCapture(video_path)
583     if not cap.isOpened():
584         raise SystemExit(f"cannot open video: {video_path}")
585     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
586     cap.release()
587     return n
588
589 def make_video_frame_loader(video_path: str):
590     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
591
592     ``frame_loader(index)`` -> BGR uint8 ndarray`` decodes the video forward-only with one
593     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
594     detector's sliding-window order); it reads sequentially and only uses a frame-seek
595     to
596     skip forward when a *resume* starts past the current position. Each ``cap.read()``
597     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
598     would
599     yield (PNG is lossless), so the detector sees identical pixels to the disk path.

```

```

598
599     Returns the count hint from the container so the caller can build the frame list;
600     ``release`` closes the capture.
601     """
602     import cv2
603     cap = cv2.VideoCapture(video_path)
604     if not cap.isOpened():
605         raise SystemExit(f"cannot open video: {video_path}")
606     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
607     state = {"pos": 0} # next index cap.read() will return
608
609     def frame_loader(index: int):
610         index = int(index)
611         if index < state["pos"]:
612             raise RuntimeError(
613                 f"streaming decode is forward-only; got index {index} after
{state['pos']}")
614         if index > state["pos"]:
615             # Forward skip only happens when resuming past cached windows. Seek once.
616             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
617             state["pos"] = index
618         ok, frame = cap.read()
619         if not ok:
620             raise RuntimeError(f"failed to decode frame {index} from {video_path}")
621         state["pos"] += 1
622         return frame
623
624     def release():
625         cap.release()
626
627     return frame_loader, count_hint, release
628
629
630     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
631                             limit: int = 0, batch_size: int = 8,
632                             log_every_batches: int = 50, cache_dir: "str | None" = None,
633                             flush_every: int = 1000, fresh: bool = False) -> str:
634         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
635         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
636         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
637         ``run``'s docstring for why.
638         """
639         frame_loader, count_hint, release = make_video_frame_loader(video_path)
640         try:
641             if count_hint <= 0:
642                 raise SystemExit(
643                     f"could not determine frame count for {video_path}; cannot stream "
644                     f"(fall back to frame extraction with --frames_dir)")
645             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
646             source_key = "video:" + osp.abspath(video_path)
647             return run(frame_list, weights, out_csv, sport, limit,
648                       batch_size=batch_size, num_workers=0,
649                       log_every_batches=log_every_batches, cache_dir=cache_dir,
650                       flush_every=flush_every, fresh=fresh,
651                       frame_loader=frame_loader, source_key=source_key)
652         finally:
653             release()
654
655
656     def main():
657         ap = argparse.ArgumentParser()
658         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
659         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")

```

```

660         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
661         ap.add_argument("--stream-video", action="store_true",
662             help="FAST PATH: decode --video on the fly and feed frames straight
to "
663                 "the detector (no PNG extraction). Output-equivalent to the
disk path.")
664         ap.add_argument("--weights", required=True)
665         ap.add_argument("--out", required=True)
666         ap.add_argument("--sport", default="badminton")
667         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
668         ap.add_argument("--batch-size", type=int, default=8)
669         ap.add_argument("--num-workers", type=int, default=4)
670         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671         ap.add_argument("--cache-dir", default=None,
672             help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673         ap.add_argument("--flush-every", type=int, default=1000,
674             help="fsync the detector cache every N windows (bounds reboot
loss)")
675         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676         args = ap.parse_args()
677
678         if args.stream_video:
679             if not args.video:
680                 raise SystemExit("--stream-video requires --video")
681             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                 batch_size=args.batch_size,
683                 log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
684                 flush_every=args.flush_every, fresh=args.fresh)
685             return
686
687         frames_dir = args.frames_dir
688         if args.video:
689             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690         if not frames_dir:
691             raise SystemExit("provide --frames_dir or --video")
692         run(frames_dir, args.weights, args.out, args.sport, args.limit,
693             batch_size=args.batch_size, num_workers=args.num_workers,
694             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695             flush_every=args.flush_every, fresh=args.fresh)
696
697
698     if __name__ == "__main__":
699         main()
700

```

5. assistant (2026-06-19T16:57:21.908Z)

I have read both files completely. Let me verify the key claims about cache persistence by examining the cleanup logic carefully.

The claim has several parts. Let me check each against the code:

****1. What `wasb\_runner.py` cleanup deletes on success** (lines 184-194):**

- The success-path cleanup builds `stale = [staged\_video]` (streaming) or `[frames\_dir, staged\_video]` (disk path), then `self.wsl\_rm\_rf(stale)`.
- The detector cache (`det\_raw.jsonl` / the `\_wasbcache/` dir) is NOT in that list. Only the staged video copy and the decoded PNG frame cache are removed.

****2. Where the cache lives and how it's keyed**** (``wasb_infer.py``):

- ``default_cache_dir`` (lines 84-88): ``<out>_wasbcache`` next to the output CSV.
- ``_DET_FILE = "det_raw.jsonl"`` (line 52), ``_cache_paths`` (lines 80-81).
- Keyed by ``source_key`` or abspath of frames dir; invalidated via ``manifest_compatible`` if frames_dir/weights/sport/window/frame-count change (lines 131-142).

****3. The on-disk JSONL record format**** (lines 185-188, 499-506, 101-104):

- Each line is one window record: ``{"w": <window_index>, "f": [[fid, [[x,y,score,scale], ...]], ...}]``, written compact (``separators=(",",":")``).
- ``_det_plain`` (lines 101-104): each candidate is ``[float x, float y, float score, int scale]``.

One subtlety to verify: the claim says the cache "persists after a successful run." The runner never deletes the cache dir on success, but does ``wasb_infer.run`` itself delete it? Let me confirm there's no cache deletion at end of a successful ``run``.

6. user (2026-06-19T16:57:23.955Z)

```
backend\pipeline\detectors\wasb_infer.py:145: def reset_cache(cache_dir: str) -> None:
backend\pipeline\detectors\wasb_infer.py:150:         os.remove(p)
backend\pipeline\detectors\wasb_infer.py:438:         reset_cache(cache_dir)
backend\pipeline\detectors\base.py:129:     def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
backend\pipeline\detectors\base.py:136:         self._wsl_bash(f"rm -rf {quoted}")
backend\pipeline\detectors\tracknet_runner.py:209:         self._wsl_rm_rf([wsl_video])
backend\pipeline\detectors\wasb_runner.py:194:         self._wsl_rm_rf(stale)
```

7. assistant (2026-06-19T16:57:26.427Z)

Let me confirm the only ``reset_cache`` call site (line 438) is on the incompatible/fresh branch, not the success path, and check ``_wsl_rm_rf`` in base.py.

8. user (2026-06-19T16:57:27.328Z)

```
125         return int((res.stdout or "").strip()) / (1024 ** 3)
126     except (ValueError, AttributeError):
127         return None
128
129     def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
130         """Best-effort recursive delete of WSL paths (post-success cleanup; never
raises)."""
131         paths = [p for p in wsl_paths if p]
132         if not paths:
133             return
134         quoted = " ".join(wsl_tilde_quote(p) for p in paths)
135         try:
136             self._wsl_bash(f"rm -rf {quoted}")
137         except (subprocess.TimeoutExpired, OSError) as e:
138             logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
139
```

9. assistant (2026-06-19T16:57:33.467Z)

Confirmed. The findings are fully consistent. Let me note one important nuance about cache location for the production runner path.

The cache dir is ``<out>_wasbcache``, where ``--out`` is ``wsl_out_dir + '/' + key + '_wasb.csv'`` (wasb_runner.py line 168), and ``wsl_out_dir = to_wsl_mnt_path(output_win_dir)`` (line 133). So in the production WSL run, the cache is written under the ``/mnt/...`` translation of the Windows ``output_win_dir`` ? meaning it lands on the Windows filesystem next to the CSV, persisting on Windows after the WSL process exits. The ``reset_cache`` at line 438 only fires on the fresh/incompatible branch (line 435-439), never on the success path. So the cache survives a successful run.

Now I have everything verified. Let me record the structured verdict.

10. user (2026-06-19T16:57:51.845Z)

Structured output provided successfully

11. assistant (2026-06-19T16:58:03.413Z)

Verified all parts of the claim. Summary:

****VERIFIED**** ? The raw detector candidate cache `det\_raw.jsonl` persists after a successful WASB run and supports GPU-free offline replay.

Key evidence:

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb\_runner.py:189-194` ? success cleanup deletes only the staged video copy and (off the streaming path) the PNG frame cache via `\_wsl\_rm\_rf(stale)`. The detector cache is never in `stale`.
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb\_infer.py:145-151,435-438` ? `reset\_cache` (the only deleter) runs solely on the fresh/incompatible branch, never on the success path; `run()` ends at :540 writing the CSV with no cache deletion.
- Offline replay: `wasb\_infer.py:453-454,529-530` skip the GPU detector pass when the cache is complete and re-run only the cheap tracker over the full per-window candidate set rebuilt from `det\_raw.jsonl`.
- Location/keying: `- Record format: one compact JSON object per line, `{ "w":<window idx>,"f":[[fid,[[x,y,score,scale],...]],...]} ` ? candidates are `[float x, float y, float score, int scale]` (`_det_plain` :101-104, payload built :499-506).

One nuance worth carrying into the doc: the cache only survives across runs because production sets `--out` to the `/mnt`-translated Windows output dir (`wasb\_runner.py:133,168`), so `